POGLAVLJE 2

ANALOGNI RAČUNARI

Eldin Begić

2.1 Uvod

Razvoj računarskih sistema tokom historije nije bio linearan niti isključivo vezan za digitalne tehnologije kakve danas poznajemo. Prije pojave savremenih digitalnih računara, značajnu ulogu u nauci, inženjerstvu i vojnoj industriji imali su analogni računari, koji su omogućavali rješavanje složenih matematičkih problema korištenjem fizičkih procesa. Analogni računari predstavljaju klasu računarskih sistema koji umjesto diskretnih numeričkih vrijednosti koriste kontinuirane fizičke veličine, kao što su električni napon, struja, mehaničko pomjeranje ili rotacija, za modeliranje i rješavanje matematičkih relacija.

Za razliku od digitalnih računara, koji podatke obrađuju u binarnom obliku i izvršavaju instrukcije zapisane u obliku programa, analogni računari funkcionišu na principu direktne analogije između matematičkog modela i fizičkog sistema. Drugim riječima, matematičke jednačine koje opisuju neki sistem preslikavaju se u električne ili mehaničke strukture čije ponašanje u vremenu odgovara rješenju tih jednačina. Ovakav pristup omogućavao je izuzetno brzo rješavanje diferencijalnih jednačina, što je bilo od presudnog značaja u oblastima poput balistike, aerodinamike, kontrole procesa i simulacije dinamičkih sistema.

Historijski gledano, prvi oblici analognog računanja javljaju se još u antičkom periodu, a jedan od najpoznatijih primjera je Antikitera mehanizam, mehanički uređaj korišten za astronomska predviđanja. Tokom 19. i početkom 20. stoljeća razvijeni su mehanički i elektromehanički analogni računari, uključujući i diferencijalne analizatore koji su se koristili za rješavanje sistema diferencijalnih jednačina. Pravi procvat analognih računara nastupa sredinom 20. stoljeća pojavom elektronskih analognih računara, koji koriste operacione pojačavače, otpornike, kondenzatore i druge elektronske komponente za realizaciju matematičkih operacija.

Jedna od ključnih karakteristika analognih računara jeste način na koji se vrši njihovo programiranje. Za razliku od digitalnih sistema, kod kojih se programiranje obavlja pisanjem koda u formalnim programskim jezicima, analogni računari se programiraju putem ožičavanja funkcionalnih blokova. Ovaj način programiranja, poznat kao ožičeno programiranje, podrazumijeva fizičko povezivanje modula koji realizuju osnovne matematičke operacije, poput sabiranja, množenja, integracije i inverzije signala. Na taj način, sama struktura veze između komponenti predstavlja program, dok vrijednosti konstanti i početni uslovi određuju konkretno ponašanje sistema.

Iako su digitalni računari vremenom gotovo u potpunosti potisnuli analogne računare zbog svoje fleksibilnosti, preciznosti i mogućnosti višestruke namjene, principi analognog računa-

nja i dalje imaju značajnu teorijsku i praktičnu vrijednost. Posebno su interesantni sa aspekta programskih jezika i prevodilaca, jer predstavljaju alternativni pogled na pojam programiranja, gdje se apstraktni matematički modeli direktno mapiraju u fizičku realizaciju, bez posredovanja tekstualnog koda, kompajlera ili interpretatora.

Cilj ovog seminarskog rada je da se detaljno predstavi koncept ožičenog programiranja analognih računara, uz historijski pregled njihovog razvoja, objašnjenje principa rada osnovnih funkcionalnih blokova, kao i prikaz konkretnih primjera programiranja i rješavanja diferencijalnih jednačina. Poseban akcenat biće stavljen na razumijevanje analognih računara kao specifične forme računarskog sistema i njihovog odnosa prema savremenim konceptima programiranja i obrade informacija.



Slika 2.1: Historijski primjer analognog računanja (Antikitera mehanizam).

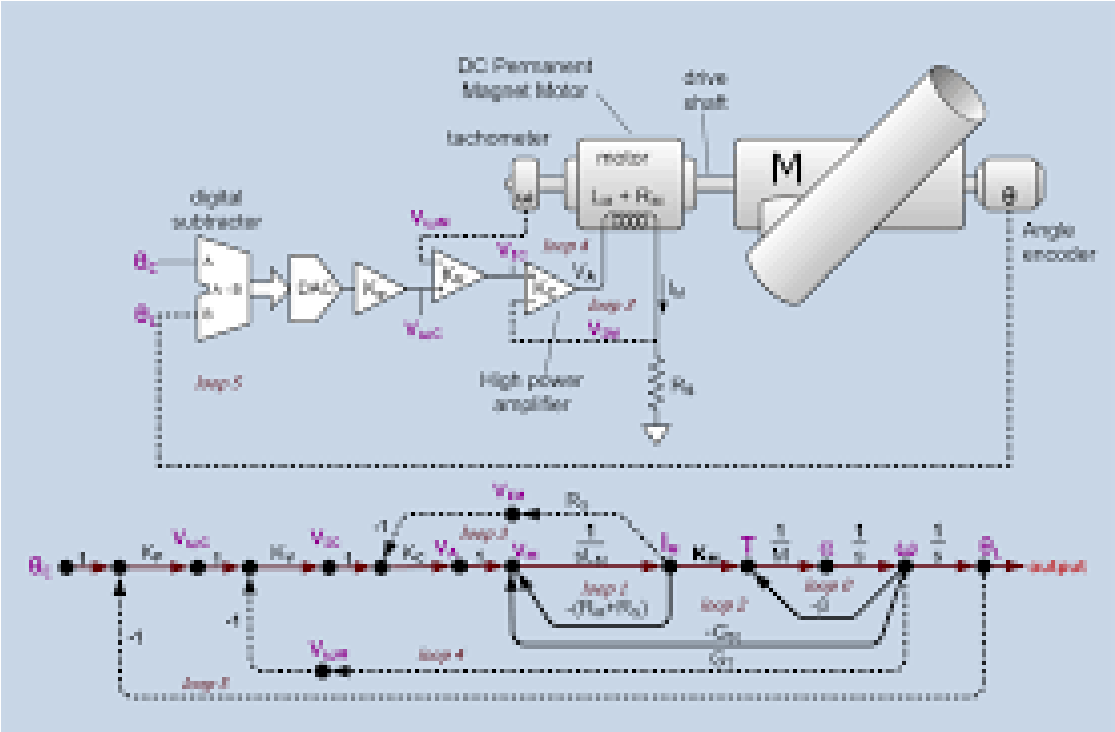
2.2 Princip rada analognih računara

2.2.1 Realizacija matematičkog modela

Osnovni princip rada analognog računara zasniva se na ideji da se matematički problem može direktno preslikati u fizički sistem čije ponašanje odgovara rješenju tog problema. U elektronskim analognim računarima, fizička veličina koja se najčešće koristi za predstavljanje matematičkih varijabli jeste električni napon, dok se promjena napona u vremenu tumači kao promjena vrijednosti varijable.

Za razliku od digitalnih računara, koji izvršavaju diskretne instrukcije nad numeričkim vrijednostima, analogni računar ne izvršava niz naredbi, već se ponaša kao kontinuirani dinamički sistem. Matematičke relacije, posebno diferencijalne jednačine, realizuju se pomoću elektronskih sklopova čija unutrašnja dinamika direktno odgovara matematičkim operacijama koje se modeliraju.

U tom smislu, analogni računar se može posmatrati kao specijalizovani hardverski simulator matematičkog sistema, gdje vrijeme ima stvarno fizičko značenje, a rješenje se dobija u realnom vremenu.



Slika 2.2: Preslikavanje matematičkog modela u fizičku realizaciju analognog računara.

2.2.2 Pojam programa u analognom računarstvu

U analognom računarstvu pojam programa značajno se razlikuje od klasičnog shvatanja programa u digitalnim sistemima. Program ne postoji kao tekstualni zapis pisan u programskom jeziku, već kao struktura međusobno povezanih funkcionalnih blokova. Ovakav način realizacije naziva se ožičeno programiranje.

U ovom kontekstu:

- program predstavlja topologiju veza između modula,
- instrukcije su implicitno definisane ponašanjem elektronskih sklopova,
- konstante se zadaju vrijednostima otpornika i potenciometara,
- početni uslovi se postavljaju inicijalnim naponima na integratorima.

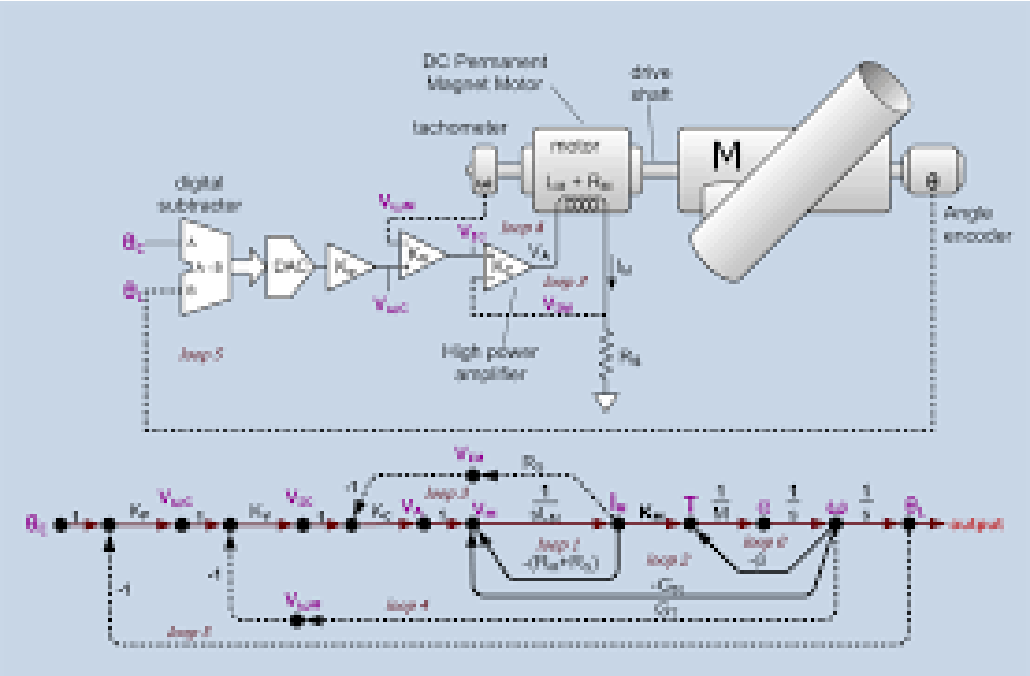
Na ovaj način, matematički model sistema ne prolazi kroz faze leksičke, sintaksne i semantičke analize, kao kod digitalnih jezika, već se direktno mapira u fizičku strukturu sistema. Ožičavanje predstavlja trajnu i eksplicitnu realizaciju semantike problema.

2.2.3 Signal-flow grafovi kao apstraktni model analognog programa

Kako bi se proces ožičenog programiranja formalnije opisao, često se koristi koncept signal-flow grafova. Signal-flow graf predstavlja apstraktan model sistema u kojem:

- čvorovi predstavljaju matematičke varijable ili operacije,
- grane predstavljaju prenos signala,
- težine grana odgovaraju konstantama množenja,
- povratne veze modeliraju zavisnosti u diferencijalnim jednačinama.

Signal-flow graf se može posmatrati kao grafički ekvivalent programa, koji se zatim direktno prevodi u fizičko ožičenje analognog računara. Ovaj pristup omogućava sistematsko projektovanje analognog programa i jasno razdvajanje matematičkog modela od njegove elektronske realizacije.



Slika 2.3: Primjer signal-flow grafa kao apstraktne reprezentacije analognog „programa“.

2.2.4 Operacioni pojačavač kao osnovni gradivni element

Centralni element elektronskog analognog računara je operacioni pojačavač. Njegove idealizovane karakteristike, kao što su vrlo veliko pojačanje, visoka ulazna impedansa i niska izlazna impedansa, omogućavaju preciznu realizaciju matematičkih operacija.

Kombinacijom operacionog pojačavača sa pasivnim komponentama moguće je realizovati osnovne funkcionalne blokove koji čine „instrukcijski skup“ analognog računara.

2.2.5 Realizacija osnovnih matematičkih funkcija

2.2.5.1 Sabirač

Sabirač omogućava zbrajanje više ulaznih signala i predstavlja osnovu za formiranje linearnih kombinacija varijabli. Najčešće se realizuje kao invertirajući sabirač.

Matematički model sabirača dat je izrazom:

$$V_{out} = -(V_1 + V_2 + \ldots + V_n)$$

Negativni predznak je posljedica načina realizacije i može se eliminisati dodatnim inverterom. Sabirači se koriste za:

- implementaciju desnih strana diferencijalnih jednačina,
- formiranje povratnih veza,
- kombinovanje više uticaja na jednu varijablu.

2.2.5.2 Inverter

Inverter je jednostavan funkcionalni blok koji mijenja predznak signala:

$$V_{out} = -V_{in}$$

Ova funkcija je neophodna za modeliranje sistema sa negativnom povratnom vezom, prigušenjem ili stabilizacijom.

2.2.5.3 Množenje konstantom

Množenje signala konstantom realizuje se pomoću otpornika ili potenciometara. Promjenom odnosa otpornika moguće je jednostavno podešavati parametre sistema bez promjene strukture ožičenja.

Matematički izraz glasi:

$$V_{out} = k \cdot V_{in}$$

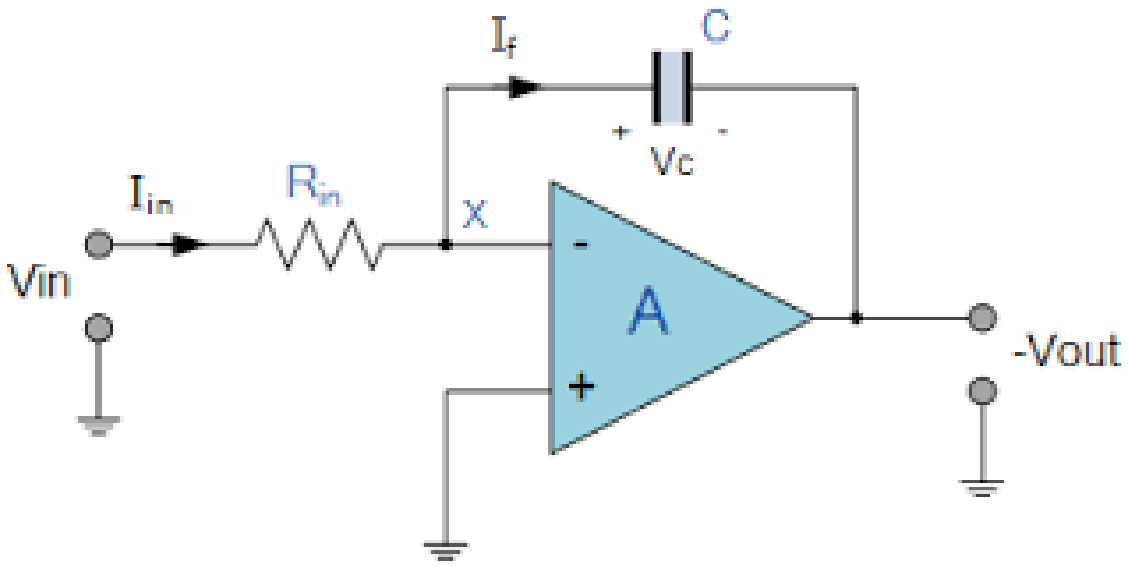
Ovaj blok omogućava direktnu implementaciju koeficijenata u diferencijalnim jednačinama koje opisuju fizičke sisteme.

2.2.5.4 Integrator

Integrator predstavlja ključni element analognog računara jer omogućava direktno rješavanje diferencijalnih jednačina. Njegova funkcija je:

$$V_{out}(t) = -\frac{1}{RC} \int V_{in}(t)\, dt$$

Povezivanjem više integratora moguće je realizovati sisteme diferencijalnih jednačina višeg reda, što čini analogni računar posebno pogodnim za simulaciju dinamičkih sistema.



Slika 2.4: Osnovni funkcionalni blokovi analognog računara.

2.2.6 Skaliranje i stabilnost analognog programa

Zbog fizičkih ograničenja elektronskih komponenti, vrijednosti signala u analognom računaru moraju ostati unutar dozvoljenog naponskog opsega. Iz tog razloga neophodno je izvršiti skaliranje matematičkih jednačina prije njihove realizacije.

Skaliranje ima sljedeće ciljeve:

- sprječavanje zasićenja pojačavača,
- povećanje numeričke stabilnosti,
- poboljšanje tačnosti rješenja.

Ovaj postupak je analogan normalizaciji i kondicioniranju problema u numeričkim metodama digitalnog računanja.

2.2.7 Analogni program kao semantička realizacija problema

Za razliku od digitalnih programa, gdje postoji jasna razlika između sintakse i semantike, u analognom računarstvu semantika problema je direktno ugrađena u hardver. Ožičavanje sistema predstavlja konačnu realizaciju značenja matematičkog modela, dok ponašanje sistema u vremenu predstavlja izvršavanje programa.

Na ovaj način, analogni računar pruža jedinstven pogled na koncept programiranja, gdje se apstraktni matematički modeli neposredno realizuju kroz fizičke procese.

2.3 Diferencijalne jednačine

2.3.1 Uloga diferencijalnih jednačina

Diferencijalne jednačine predstavljaju osnovni matematički alat za opisivanje dinamičkih sistema u fizici, elektrotehnici, mehanici i kontroli procesa. Analogni računari su historijski razvijani upravo sa ciljem da efikasno rješavaju ovakve jednačine, jer njihova struktura prirodno odgovara matematičkom obliku diferencijalnih relacija.

Za razliku od digitalnih računara, koji rješenja diferencijalnih jednačina dobijaju numeričkim aproksimacijama u diskretnim vremenskim koracima, analogni računari koriste integratore kao osnovne građevne blokove, čime se diferencijalne jednačine rješavaju kontinuirano u realnom vremenu. Zbog toga se analogno računanje može posmatrati kao direktna fizička realizacija matematičkog modela sistema.

2.3.2 Sistematski postupak programiranja diferencijalnih jednačina

Programiranje diferencijalnih jednačina na analognom računaru ne vrši se intuitivno, već slijedi jasno definisan i sistematski postupak. Ovaj postupak se može posmatrati kao analogni ekvivalent algoritma u digitalnom programiranju.

Korak 1: Definisanje matematičkog modela

Proces započinje definisanjem diferencijalne jednačine ili sistema jednačina koji opisuje posmatrani fizički sistem. Na primjer, razmatra se jednačina prvog reda:

$$\frac{dy(t)}{dt} = -a\,y(t)$$

gdje je a pozitivna konstanta sistema.

Korak 2: Prevođenje jednačine u integracioni oblik

Kako analogni računari direktno realizuju integraciju, diferencijalna jednačina se mora preurediti u oblik pogodan za integratore. Prethodna jednačina se zapisuje kao:

$$y(t) = \int -a\,y(t)\,dt$$

Ovaj korak predstavlja ključni prelaz sa diferencijalnog na integralni oblik, koji omogućava implementaciju pomoću analognog hardvera.

Korak 3: Identifikacija potrebnih funkcionalnih blokova

Na osnovu integracionog oblika, identifikuju se osnovni blokovi:

- jedan integrator za računanje $y(t)$,

- jedan blok za množenje konstantom a,
- jedan inverter za realizaciju negativnog predznaka,
- sabirač (ako postoji više članova).

Ovi blokovi predstavljaju „instrukcije“ analognog programa.

Korak 4: Konstrukcija signal-flow grafa

Signal-flow graf predstavlja apstraktni prikaz odnosa između varijabli i funkcionalnih blokova. U ovom grafu:

- čvor predstavlja varijablu $y(t)$,
- grana sa težinom a predstavlja množenje konstantom,
- povratna veza implementira zavisnost izlaza od samog sebe.

Signal-flow graf služi kao međukorak između matematičkog modela i fizičkog ožičenja.

Korak 5: Fizičko ožičenje analognog računara

Na osnovu signal-flow grafa vrši se fizičko povezivanje modula:

- izlaz integratora predstavlja varijablu $y(t)$,
- taj izlaz se vodi kroz množač konstantom a,
- zatim kroz inverter,
- i vraća se na ulaz integratora.

Ovim ožičenjem ostvaruje se potpuna realizacija matematičke jednačine u hardveru.

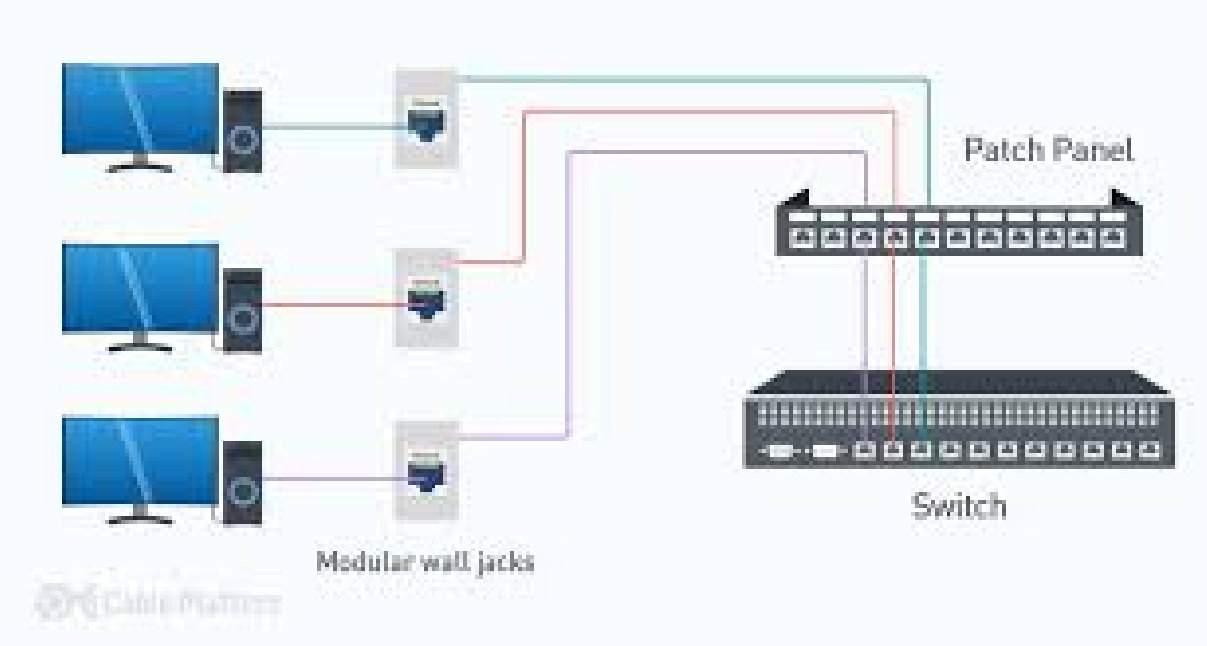
Korak 6: Postavljanje početnih uslova

Početni uslovi diferencijalne jednačine realizuju se inicijalnim punjenjem kondenzatora u integratoru. Vrijednost početnog napona odgovara početnoj vrijednosti varijable $y(0)$.

Ovaj korak je analogan inicijalizaciji varijabli u digitalnom programu.

Korak 7: Skaliranje i provjera stabilnosti

Prije puštanja sistema u rad, vrši se skaliranje signala kako bi se spriječilo zasićenje komponenti i osigurala stabilnost rješenja. Ovaj korak je posebno važan kod sistema sa povratnom vezom.



Slika 2.5: Primjer ožičenog programiranja pomoću patch panela – shema povezivanja funkcionalnih blokova

2.3.3 Programiranje diferencijalnih jednačina višeg reda

Većina realnih sistema opisana je diferencijalnim jednačinama višeg reda. Takve jednačine se prvo transformišu u sistem jednačina prvog reda, koji se zatim implementira pomoću lanca integratora.

Primjer jednačine drugog reda:

$$\ddot{x}(t) + b\dot{x}(t) + cx(t) = 0$$

Uvođenjem pomoćnih varijabli:

$$x_1 = x, \quad x_2 = \dot{x}$$

dobija se sistem:

$$\dot{x}_1 = x_2$$

$$\dot{x}_2 = -bx_2 - cx_1$$

Svaka od ovih jednačina realizuje se pomoću posebnog integratora, dok se sabirači i množači koriste za formiranje desnih strana sistema.

2.3.4 Studija slučaja: dinamički sistem drugog reda

Kao tipičan primjer analognog programiranja često se razmatra sistem masa – opruga – prigušivač. Ovaj sistem se koristi jer jasno demonstrira uticaj povratne veze, prigušenja i oscilatornog ponašanja.

Analogni računar omogućava:

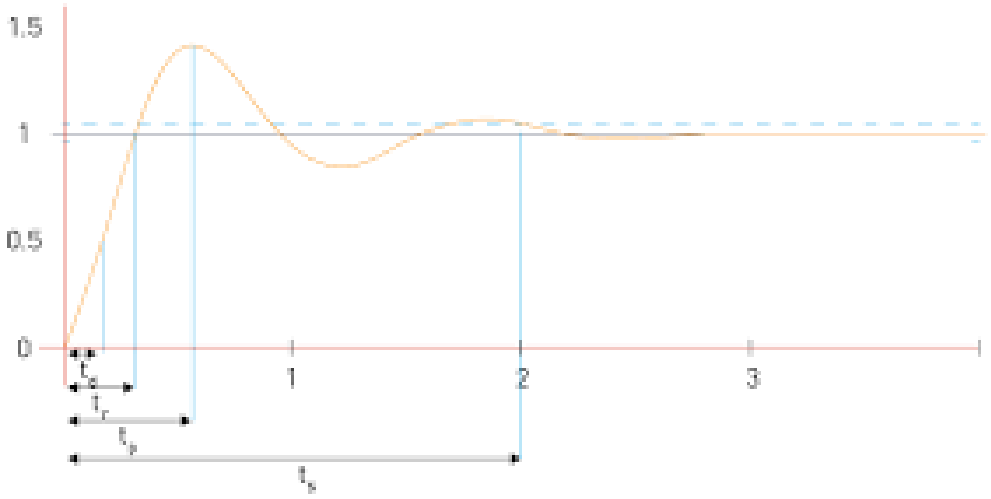
- posmatranje oscilacija u realnom vremenu,
- analizu stabilnosti sistema,
- promjenu parametara tokom rada.

Na ovaj način analogni računar postaje alat ne samo za rješavanje jednačina, već i za eksperimentalno ispitivanje ponašanja sistema.

2.3.5 Interpretacija rezultata i fizičko značenje rješenja

Rezultati analognog računanja se najčešće posmatraju pomoću osciloskopa ili zapisivača. Izlazni signal predstavlja vremenski tok rješenja diferencijalne jednačine.

Za razliku od digitalnih metoda, gdje je rezultat niz diskretnih tačaka, analogni rezultat je kontinuirana funkcija vremena, što omogućava intuitivno razumijevanje dinamike sistema.



Slika 2.6: Primjer vremenskog odziva (izlaz analognog računanja kao funkcija vremena).

2.3.6 Analogno programiranje kao algoritamski proces

Iako analogni računari ne koriste programske jezike u klasičnom smislu, proces programiranja diferencijalnih jednačina može se jasno posmatrati kao algoritamski postupak. Svaki korak – od matematičkog modela do ožičenja – odgovara fazama razvoja programa u digitalnom okruženju.

Na ovaj način, analogno programiranje predstavlja alternativni, ali konceptualno vrlo bogat pristup rješavanju računarskih problema, koji pruža dublji uvid u samu prirodu računanja.

2.4 Greške, ograničenja i stabilnost sistema

2.4.1 Neidealnosti i izvori grešaka

Iako analogni računari omogućavaju direktno i intuitivno rješavanje matematičkih modela, njihova praktična realizacija podložna je različitim izvorima grešaka. Ove greške proizlaze iz neidealnih karakteristika elektronskih komponenti i fizičkih ograničenja sistema, te imaju značajan uticaj na tačnost i pouzdanost dobijenih rješenja.

Jedan od osnovnih izvora grešaka jeste šum, koji može poticati iz elektronskih komponenti, okoline ili napajanja. Šum se superponira na korisni signal i može dovesti do fluktuacija u izlaznim vrijednostima, posebno u sistemima sa velikim pojačanjem ili osjetljivim povratnim vezama.

Drugi važan izvor grešaka predstavlja drift komponenti, odnosno promjena električnih karakteristika otpornika, kondenzatora i operacionih pojačavača tokom vremena ili usljed promjena temperature. Drift može uzrokovati postepeno odstupanje rješenja od teorijski očekivanog ponašanja, što je posebno izraženo kod dugotrajnih simulacija.

Dodatni problem predstavljaju nelinearnosti realnih komponenti. Iako se u teoriji pretpostavlja idealno linearno ponašanje operacionih pojačavača, u praksi se javljaju ograničenja poput zasićenja izlaza, ograničenog frekvencijskog opsega i konačnog pojačanja, što može narušiti tačnost matematičkog modela.

2.4.2 Ograničenja analognog računanja

Analogni računari su inherentno ograničeni fizičkim karakteristikama sistema u kojem su realizovani. Jedno od najznačajnijih ograničenja jeste ograničen dinamički opseg signala. Elektronski analogni računari najčešće rade u rasponu napona od približno ± 10 V, što nameće potrebu za pažljivim skaliranjem varijabli.

Još jedno ograničenje odnosi se na preciznost rješenja. Za razliku od digitalnih računara, gdje se preciznost može povećati povećanjem broja bitova, preciznost analognog računara zavisi od tolerancija komponenti, stabilnosti napajanja i kvaliteta mjernih uređaja. Zbog toga analogni računari nisu pogodni za probleme koji zahtijevaju visoku numeričku preciznost.

Također, analogni računari su po svojoj prirodi namjenski sistemi. Promjena problema zahtijeva fizičku promjenu ožičenja, što smanjuje fleksibilnost u odnosu na digitalne sisteme koji koriste programski kod kao apstraktni sloj.

2.4.3 Akumulacija grešaka u sistemima sa integratorima

Integrator je ključni element analognog računara, ali ujedno i jedan od glavnih izvora akumulacije grešaka. Male greške na ulazu integratora mogu se vremenom akumulirati, što dovodi do značajnog odstupanja izlaznog signala.

Posebno su problematični sistemi sa dugim vremenskim konstantama ili sa slabom povratnom stabilizacijom, gdje se greške ne prigušuju, već se postepeno povećavaju. Ovaj fenomen je analogan numeričkoj nestabilnosti kod digitalnih metoda integracije.

2.4.4 Stabilnost analognog programa

Stabilnost predstavlja jedan od najvažnijih aspekata programiranja analognog računara. Sistem se smatra stabilnim ako male promjene ulaznih signala ili početnih uslova ne dovode do nekontrolisanog rasta izlaznih veličina.

Stabilnost analognog programa zavisi prvenstveno od:

- strukture povratnih veza,
- vrijednosti parametara sistema,
- načina skaliranja jednačina.

Sistemi sa negativnom povratnom vezom uglavnom imaju stabilno ponašanje, dok sistemi sa pozitivnom povratnom vezom mogu dovesti do eksponencijalnog rasta signala i zasićenja komponenti.

2.4.5 Uticaj skaliranja na stabilnost

Skaliranje ne utiče samo na amplitudu signala, već i na stabilnost cijelog sistema. Neadekvatno skaliranje može uzrokovati:

- zasićenje operacionih pojačavača,
- gubitak linearnosti,
- pojavu oscilacija koje ne postoje u originalnom matematičkom modelu.

Zbog toga se skaliranje može posmatrati kao sastavni dio procesa programiranja, a ne samo kao tehnička korekcija.

2.4.6 Poređenje stabilnosti analognog i digitalnog računanja

U digitalnim sistemima stabilnost rješenja diferencijalnih jednačina zavisi od izbora numeričke metode i veličine vremenskog koraka. Kod analognog računanja, stabilnost zavisi od fizičkih karakteristika sistema i strukture povratne veze.

Dok digitalni sistemi omogućavaju kontrolu stabilnosti putem algoritama, analogni sistemi zahtijevaju duboko razumijevanje dinamike sistema i pažljivo projektovanje ožičenja. Ovo čini analogno programiranje zahtjevnijim, ali i konceptualno bogatijim pristupom.

2.4.7 Značaj analize grešaka i stabilnosti

Analiza grešaka i stabilnosti predstavlja neizostavan dio rada sa analognim računarima. Bez ovog koraka, dobijeni rezultati mogu biti pogrešno interpretirani ili potpuno neupotrebljivi.

Razumijevanje ograničenja analognog računanja omogućava pravilnu primjenu ove tehnologije i pruža vrijedan uvid u šire principe računarskih sistema, posebno u kontekstu modeliranja i simulacije dinamičkih procesa.

2.5 Veza sa programskim jezicima

2.5.1 Širenje pojma programiranja

U savremenom računarstvu pojam programiranja najčešće se vezuje za pisanje instrukcija u formalnim programskim jezicima, koje se zatim prevode i izvršavaju na digitalnim računarima. Međutim, analogno računarstvo pokazuje da je pojam programiranja znatno širi i da se ne mora nužno oslanjati na tekstualne reprezentacije i diskretne instrukcije.

U kontekstu analognog računara, programiranje se ostvaruje putem fizičkog povezivanja funkcionalnih blokova, čime se direktno definiše ponašanje sistema. Na taj način, analogni računar predstavlja alternativni model programiranja u kojem je program struktura sistema, a ne sekvenca naredbi.

Ovakav pristup omogućava dublje razumijevanje same suštine programiranja kao procesa mapiranja apstraktnog modela u izvršivu realizaciju.

2.5.2 Analogni računar kao hardverski prevodilac

U teoriji programskih jezika, prevodilac (kompajler ili interpretator) ima zadatak da apstraktni zapis programa prevede u oblik koji je razumljiv i izvršiv na ciljanom hardveru. Analogni računar može se posmatrati kao specifična vrsta hardverskog prevodioca, gdje se matematički model direktno prevodi u fizičku strukturu sistema.

U ovom procesu:

- ulaz predstavlja matematička jednačina ili sistem jednačina,
- proces prevođenja se sastoji od projektovanja signal-flow grafa i ožičenja,
- izlaz je fizički sistem čije ponašanje odgovara rješenju problema.

Za razliku od digitalnih prevodilaca, analogni prevodilac ne generiše kod u međureprezentaciji ili mašinskom jeziku, već direktno realizuje semantiku problema u hardveru.

2.5.3 Izostanak klasičnih faza prevođenja

Kod digitalnih programskih jezika, proces prevođenja tipično se sastoji od faza kao što su leksička analiza, sintaksna analiza, semantička analiza i generisanje koda. U analognom računarstvu ove faze ne postoje u klasičnom smislu.

Međutim, njihove uloge se implicitno ostvaruju kroz druge mehanizme:

- sintaksa je zamijenjena pravilima povezivanja modula,
- semantika je ugrađena u fizičko ponašanje sklopova,
- izvršavanje programa započinje odmah po uključenju sistema.

Ovaj direktni pristup ukazuje na to da su faze prevođenja konceptualne, a ne nužno vezane za tekstualne jezike.

2.5.4 Semantika programa u analognom računarstvu

Jedan od ključnih doprinosa analognog računarstva teoriji programskih jezika jeste drugačiji pogled na pojam semantike. U digitalnim jezicima, semantika programa se definiše pravilima interpretacije instrukcija nad apstraktnim mašinama. U analognom računarstvu, semantika programa je direktno implementirana u fizičkom sistemu.

Drugim riječima, značenje programa nije opisano pravilima, već je sadržano u načinu na koji se signali ponašaju u vremenu. Ovo predstavlja ekstremni oblik tzv. operacionalne semantike, gdje se ponašanje sistema posmatra kao izvršavanje programa.

2.5.5 Signal-flow grafovi kao analog apstraktnog sintaksnog stabla

U digitalnim jezicima, apstraktno sintaksno stablo (AST) predstavlja strukturni prikaz programa. U analognom računarstvu, sličnu ulogu ima signal-flow graf, koji opisuje strukturu zavisnosti između varijabli i operacija.

Kao što se AST prevodi u mašinski kod, signal-flow graf se prevodi u fizičko ožičenje analognog računara. Ova analogija omogućava jasnu paralelu između klasičnog prevođenja programa i ožičenog programiranja.

2.5.6 Kompajliranje naspram fizičkog mapiranja

U digitalnim sistemima, kompajliranje je proces u kojem se program transformiše iz jednog oblika u drugi. U analognom računarstvu, sličan proces postoji u obliku fizičkog mapiranja matematičkog modela u elektronske komponente.

Ovaj proces uključuje:

- izbor funkcionalnih blokova,
- određivanje parametara,
- fizičko povezivanje komponenti.

Rezultat nije kod, već konkretan hardverski sistem koji izvršava dati „program“.

2.5.7 Analogno programiranje i model-driven razvoj

Analogno programiranje se može posmatrati kao rani oblik model-driven development pristupa, gdje je matematički model centralni artefakt razvoja, a implementacija predstavlja njegovu direktnu realizaciju.

Ovaj koncept je posebno značajan u savremenim softverskim praksama, gdje se sve više koristi automatsko generisanje koda na osnovu formalnih modela.

2.5.8 Pedagoški i teorijski značaj

Proučavanje analognog računarstva ima veliki pedagoški značaj za razumijevanje programskih jezika i prevodilaca. Ono omogućava studentima da:

- shvate razliku između sintakse i semantike,
- razumiju ulogu apstrakcije u programiranju,
- sagledaju programiranje kao proces mapiranja modela u izvršivu formu.

Analogni računari pružaju jedinstven uvid u prirodu računanja i predstavljaju vrijedan most između teorije i fizičke realizacije.

2.6 Programski jezici i ožičavanje

2.6.1 Kompajler ili interpreter?

Kod klasičnih digitalnih programa postoji tekstualni kod i prevodilac (kompajler ili interpretator) koji taj kod pretvara u izvršiv oblik. Kod analognog računara program nije tekst, nego fizička konfiguracija (ožičenje) funkcionalnih blokova. Zato analogno ožičavanje ne koristi kompajler/interpreter u klasičnom smislu.

Ipak, postoji analogna ideja „prevođenja“: matematički model se prvo prevede u signal-flow graf, a zatim u fizičku realizaciju na patch panelu. Ovaj postupak se može posmatrati kao hardversko mapiranje semantike problema u sistem, gdje „izvršavanje“ počinje odmah po uključivanju jer se ponašanje sklopa direktno poklapa sa modelom.

2.6.2 Formalna specifikacija ožičenja: primjer EBNF zapisa (BNF / EBNF)

Iako analogni računari nemaju programski jezik u standardnom smislu, moguće je uvesti formalni zapis kojim se opisuje ožičenje kao vrsta konfiguracionog jezika. Takav zapis ima sintaksu i može se formalizovati EBNF-om:

```
Program      ::= { Veza } ;
Veza         ::= Izvor "->" Odrediste ;
Izvor        ::= Blok "." Izlaz ;
Odrediste    ::= Blok "." Ulaz ;

Blok         ::= "INT" | "SUM" | "INV" | "GAIN" | "CONST" ;
Ulaz         ::= "in" | "in1" | "in2" ;
Izlaz        ::= "out" ;
```

Ovaj format ne služi da „pokrene“ analogni računar, nego da dokumentuje i provjeri ispravnost povezivanja. U historijskom smislu, patch panel dijagrami su bili ekvivalent izvornom kodu.

2.6.3 LL gramatike i analiza: analogija provjere ispravnosti povezivanja

LL gramatike se koriste kako bi parser mogao čitati ulaz slijeva na desno i praviti izvedbu programa. U analognoj analogiji, parser nije nužan za izvršavanje, ali je koristan za provjeru konfiguracije.

Ako gore definisani EBNF koristimo kao zapis ožičenja, tada LL(1) parser može automatski provjeriti:

- da li su imena blokova validna (npr. INT, SUM, INV),
- da li zadati ulazi/izlazi postoje,
- da li se veza pravi u ispravnom smjeru (izlaz $\rightarrow$ ulaz),
- da li postoji očita greška (npr. izlaz spojen na izlaz, nedostaje ulaz integratora).

Dakle, LL parsing ovdje ima ulogu validacije dokumentacije ožičenja, a ne izvršavanja programa.

2.6.4 Parser generator i analogna dokumentacija

Parser generator (npr. alat koji generiše parser iz gramatike) u digitalnom svijetu automatizira izradu prevodioca. U analognom svijetu sličnu ulogu su imale standardizovane šeme, blok dijagrami i patch panel nacrti: oni pojednostavljuju proces, smanjuju greške i standardizuju „sintaksu“ povezivanja.

U savremenim hibridnim sistemima ova ideja se vraća kroz CAD alate i automatsko generisanje konfiguracija, gdje se dio modela automatski mapira u hardver ili u konfiguraciju mješovitog sistema.

2.6.5 Tipovi podataka: analogni tip-sistem (opseg, jedinice i bandwidth)

U digitalnim jezicima tipovi (int, float, string) ograničavaju operacije i obezbjeđuju sigurnost. U analognim računarima osnovna vrijednost je signal (napon/struja), pa se „tip“ može posmatrati kroz:

- **opseg signala** (npr. ± 10 V) — prelazak opsega dovodi do zasićenja,
- **jedinice nakon skaliranja** (npr. 1 V = 1 m/s ili 1 V = 10 N),
- **frekvencijski opseg (bandwidth)** — ograničava koliko brzo signal može promijeniti vrijednost bez izobličenja.

Skaliranje se zato može posmatrati kao dio „tip-sigurnosti“: pogrešno skaliranje uzrokuje gubitak linearnosti i lažno ponašanje sistema.

2.7 Savremeni analogni i hibridni sistemi

2.7.1 Povratak interesa

Iako su analogni računari tokom druge polovine 20. stoljeća gotovo u potpunosti zamijenjeni digitalnim sistemima, posljednjih godina primjetan je ponovni porast interesovanja za analogno i hibridno računarstvo. Razlog za ovaj trend leži u ograničenjima savremenih digitalnih sistema, posebno kada je riječ o potrošnji energije, brzini obrade velikih količina podataka i efikasnom rješavanju određenih klasa problema.

Savremeni analogni sistemi se ne razvijaju kao zamjena za digitalne računare, već kao specijalizovani akceleratori koji dopunjuju digitalne platforme. Na ovaj način nastaje koncept hibridnih sistema, u kojima analogni i digitalni dijelovi sarađuju kako bi se postigle bolje performanse.

2.7.2 Hibridni analogno-digitalni sistemi

Hibridni sistemi kombinuju prednosti oba pristupa:

- analogni dio omogućava brzo i energetski efikasno računanje,
- digitalni dio obezbjeđuje fleksibilnost, kontrolu i visoku preciznost.

U takvim sistemima analogni moduli se često koriste za:

- rješavanje diferencijalnih jednačina,
- filtriranje signala,
- optimizacione probleme,
- simulaciju dinamičkih sistema.

Digitalni dio sistema ima ulogu nadzora, podešavanja parametara i obrade rezultata. Ovakva arhitektura podsjeća na klasične analogne računare, ali uz savremeni nivo automatizacije i kontrole.

2.7.3 Neuromorfno računarstvo

Jedno od najznačajnijih savremenih područja primjene analognih principa jeste neuromorfno računarstvo. Neuromorfni sistemi pokušavaju da oponašaju način rada bioloških neuronskih mreža koristeći analogne ili mješovite analogno-digitalne sklopove.

U ovim sistemima:

- neuroni se realizuju kao analogni elementi,
- sinaptičke veze predstavljaju ponderisane signale,

• računanje se odvija paralelno i kontinuirano.

Zahvaljujući analognim principima, neuromorfni sistemi postižu izuzetno nisku potrošnju energije i veliku efikasnost u zadacima kao što su prepoznavanje obrazaca, obrada signala i učenje.

2.7.4 Akceleratori i specijalizovani hardver

Savremeni istraživački i industrijski sistemi sve češće koriste analogne akceleratore za rješavanje specifičnih problema. Ovi akceleratori su dizajnirani za:

• linearne algebarske operacije,
• optimizaciju,
• simulaciju fizičkih procesa.

Analogni akceleratori se često integrišu sa digitalnim procesorima, gdje analogni dio obavlja intenzivne matematičke operacije, dok digitalni dio upravlja procesom i osigurava tačnost rezultata.

2.7.5 FPGA i analogni moduli

U modernim sistemima često se koristi kombinacija FPGA tehnologije i analognih modula. FPGA omogućava fleksibilnu digitalnu kontrolu, dok analogni moduli realizuju kontinuirane operacije.

Ovakav pristup omogućava:

• dinamičko rekonfigurisanje sistema,
• brzu promjenu parametara,
• kombinaciju diskretne i kontinuirane obrade podataka.

Na ovaj način, ideja ožičenog programiranja dobija novu formu, gdje se fizičko povezivanje zamjenjuje programabilnim strukturama.

2.7.6 Analogno računarstvo i AI

Savremeni sistemi vještačke inteligencije, posebno oni zasnovani na neuronskim mrežama, sve češće koriste analogne principe kako bi se povećala energetska efikasnost. Analogni sklopovi omogućavaju implementaciju velikog broja paralelnih operacija uz minimalnu potrošnju energije.

Ovaj pristup je posebno značajan u ugrađenim sistemima i mobilnim uređajima, gdje su resursi ograničeni.

2.7.7 Ograničenja savremenih analognih sistema

Uprkos ponovnom interesovanju, savremeni analogni sistemi i dalje imaju određena ograničenja. Problemi vezani za tačnost, šum i skalabilnost ostaju izazov, zbog čega se analogni sistemi najčešće koriste u kombinaciji sa digitalnim komponentama.

Ovo potvrđuje da analogno računarstvo danas ima ulogu komplementarne tehnologije, a ne univerzalnog rješenja.

2.7.8 Značaj savremenih analognih i hibridnih sistema

Savremeni analogni i hibridni sistemi predstavljaju most između klasičnih analognih računara i modernih digitalnih tehnologija. Oni pokazuju da osnovni principi analognog računanja i ožičenog programiranja i dalje imaju praktičnu vrijednost, posebno u oblastima gdje su brzina, paralelizam i energetska efikasnost ključni faktori.

Ovo potvrđuje da analogno računarstvo nije samo historijska etapa u razvoju računara, već i važan dio savremenih i budućih računarskih sistema.

2.8 Vizualni elementi i analogni računari

2.8.1 Značaj vizualnih prikaza

Vizualni elementi imaju izuzetno važnu ulogu u razumijevanju i analizi rada analognih računara. S obzirom na to da se analogno programiranje zasniva na fizičkom povezivanju funkcionalnih blokova, grafički prikazi omogućavaju jasnu interpretaciju odnosa između matematičkog modela i njegove realizacije u hardveru.

Za razliku od digitalnog programiranja, gdje se struktura programa često vidi kroz tekstualni kod, u analognom računarstvu struktura programa je prostorna i vizualna. Dijagrami i grafički prikazi omogućavaju da se program „vidi“, a ne samo opiše.

2.8.2 Blok dijagrami funkcionalnih modula

Blok dijagrami predstavljaju osnovni oblik vizualizacije u analognom računarstvu. U njima se svaki funkcionalni modul (sabirač, integrator, inverter, množač konstantom) prikazuje kao zaseban blok sa jasno označenim ulazima i izlazima.

Blok dijagrami omogućavaju:

- pregled strukture analognog programa,
- identifikaciju toka signala,
- razumijevanje zavisnosti između varijabli,
- lakše otkrivanje grešaka u ožičenju.

Ovi dijagrami su posebno korisni u fazi projektovanja sistema, jer omogućavaju apstraktan prikaz bez ulaska u detalje elektronske realizacije.

2.8.3 Signal-flow grafovi kao programska reprezentacija

Signal-flow grafovi predstavljaju jedan od najvažnijih vizualnih alata u analognom programiranju. Oni grafički prikazuju matematičke relacije između varijabli i operacija, te se mogu posmatrati kao analog apstraktnog sintaksnog stabla u digitalnim programskim jezicima.

U signal-flow grafu:

- čvorovi predstavljaju varijable sistema,
- grane predstavljaju matematičke operacije,
- težine grana odgovaraju konstantama,
- povratne veze predstavljaju dinamičke zavisnosti.

Ovakav graf služi kao međukorak između matematičkog modela i fizičkog ožičenja, čime se značajno olakšava proces programiranja.

2.8.4 Sheme ožičenja (patch panel dijagrami)

Sheme ožičenja predstavljaju najkonkretniji oblik vizualnog prikaza analognog programa. One direktno prikazuju kako su funkcionalni moduli fizički povezani pomoću kablova.

Ovi dijagrami:

- predstavljaju konačnu realizaciju programa,
- omogućavaju verifikaciju ispravnosti povezivanja,
- služe kao dokumentacija analognog programa.

U historijskim analognim računarima, patch panel dijagrami su bili ekvivalent izvornog koda u digitalnim sistemima.

2.8.5 Dijagrami vremenskog odziva

Rezultati analognog računanja često se prikazuju pomoću dijagrama vremenskog odziva, gdje se izlazni signal posmatra kao funkcija vremena. Ovi dijagrami omogućavaju intuitivno razumijevanje dinamike sistema.

Kroz vremenske dijagrame moguće je:

- analizirati stabilnost sistema,
- uočiti oscilatorno ponašanje,
- procijeniti uticaj parametara na odziv.

Ovakvi prikazi su posebno važni kod sistema opisanih diferencijalnim jednačinama.

2.8.6 Vizualizacija grešaka i ograničenja

Vizualni elementi se koriste i za ilustraciju grešaka i ograničenja analognog računanja. Na primjer, dijagrami mogu prikazati:

- zasićenje signala,
- uticaj šuma,
- drift integratora,
- odstupanja od idealnog modela.

Ovi prikazi omogućavaju jasnije razumijevanje razlika između teorijskog modela i praktične realizacije.

2.8.7 Uloga vizualnih elemenata u dokumentaciji i edukaciji

Vizualni elementi imaju ključnu ulogu u dokumentovanju analognog programa. Bez grafičkih prikaza, opis ožičenja i ponašanja sistema bio bi teško razumljiv i neprecizan.

U edukativnom kontekstu, vizualni prikazi pomažu studentima da:

- lakše povežu matematiku sa hardverom,
- razumiju tok signala,
- shvate koncept ožičenog programiranja.

Zbog toga se vizualni elementi smatraju sastavnim dijelom ozbiljnog rada u oblasti analognog računarstva.

2.8.8 Vizualni elementi kao dio inženjerskog razmišljanja

Upotreba dijagrama i grafičkih prikaza u analognom računarstvu odražava inženjerski pristup rješavanju problema. Umjesto apstraktnih simbola, problemi se rješavaju kroz strukture koje se mogu vidjeti, analizirati i fizički realizovati.

Na taj način, vizualni elementi ne predstavljaju samo pomoćno sredstvo, već ključni alat za razumijevanje i programiranje analognih računara.

2.9 Zaključak

Analogni računari predstavljaju jednu od najranijih, ali i konceptualno najdubljih faza u razvoju računarskih sistema. Iako su danas u velikoj mjeri potisnuti digitalnim tehnologijama, njihovi principi rada i način programiranja nude vrijedan uvid u samu prirodu računanja. Kroz ožičeno programiranje, analogni računari pokazuju da program ne mora nužno biti zapisan u tekstualnom obliku, već može postojati kao fizička struktura koja direktno realizuje matematički model.

U ovom radu je prikazan historijski razvoj analognih računara, princip njihovog rada, kao i realizacija osnovnih funkcionalnih blokova koji omogućavaju rješavanje diferencijalnih jednačina. Poseban akcenat stavljen je na proces programiranja diferencijalnih jednačina.

2.10 Bibliografija

[1] Wikipedia contributors, *Analog computer*, Wikipedia – The Free Encyclopedia. Dostupno na: *https://en.wikipedia.org/wiki/Analog_computer* (pristupljeno: decembar 2025.)

[2] G. A. Korn, T. M. Korn, *Handbook of Analog Computation*, McGraw-Hill, 1958. Dostupno na: *https://www.analogmuseum.org/library/handbook_of_analog_computation.pdf* (pristupljeno: decembar 2025.)

[3] The Analog Museum, *Analog Computing – Historical Resources*. Dostupno na: *https://www.analogmuseum.org* (pristupljeno: decembar 2025.)